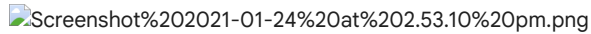


## ✓ Author: Dr Mike Lakoju, CardiffMet

Screenshot%202021-01-24%20at%202.53.10%20pm.png

## ✓ Import Library

```
import numpy as np
```

## ✓ Define "Inputs, outputs and weights" as Numpy arrays

## ✓ Inputs

```
# Creating input values as a matrix not as a vector
inputs = np.array([[0,0], [0,1], [1,0], [1,1]])
```

```
# Chcking the shape of the inputs
inputs.shape
```

```
(4, 2)
```

## ✓ Outputs

```
outputs = np.array([0, 0, 0, 1])
```

```
#Checking the shape of the outputs
```

```
outputs.shape
```

```
(4,)
```

## ✓ Weights

```
# one weight for x1 and one for x2
weights = np.array([0.0, 0.0])
```

## ✓ Learning Rate

```
learning_rate = 0.1
```

## ✓ Step function

```
# This is our Activation function

def step_function(sum):
    if (sum >= 1):
        #print(f'The Sum of Weights is Greater or equal to 1')
        return 1
    else:
        #print(f'The Sum of Weights is NOT > or = to 1')
        return 0
```

## ✓ Process Output

**We define a function that allows us to calculate/ process the output. The function accepts an instance of our data, then calculate the sum function using Numpy. Finally, we check the output by passing it through the "Step Function."**

```
def cal_output(instance):
    sum_func = instance.dot(weights)
    return step_function(sum_func)
```

We pass it as alist in a numpy array ...

```
cal_output(np.array([[1,1]]))
```

```
0
```

## ▼ Train

```
# Check the number of outputs
```

```
len(outputs)
```

```
4
```

```
# Checking the index of the input at postion 3 ..  
# this is the last inpute value  
inputs[3]
```

```
array([1, 1])
```

```
inputs
```

```
array([[0, 0],  
       [0, 1],  
       [1, 0],  
       [1, 1]])
```

**Note that:** usually, we will need to define the number of epochs, because we will never really get a value of zero when dealing with real-world data. However, for this small data, we will run the loop till we obtain zero error

```
def train():  
    #  
    total_error_value = 1  
    # While the total_error_value is not equal to zero. we are asumming that at the start of running our network  
    while (total_error_value != 0):  
        #making the total_error 0 so we can do other calculations  
        total_error_value = 0  
        #Looping into each row of the dataset (remember indexing in python starts at zero hence 0-3 which are 4  
        for i in range(len(outputs)):  
            #Calculating predictions  
            prediction = cal_output(inputs[i])  
            # Calculating the absolute value of the error  
            error = abs(outputs[i] - prediction)  
            #Updating the error  
            total_error_value += error  
  
            if error > 0:  
                for j in range(len(weights)):  
                    #updating the weights for x1 and x2  
                    weights[j] = weights[j] + (learning_rate * inputs[i][j] * error)  
                    print('Weight updated to: ' + str(weights[j]))  
        print('Total error Value: ' + str(total_error_value))
```

```
train()
```

```
Weight updated to: 0.1  
Weight updated to: 0.1  
Total error Value: 1  
Weight updated to: 0.2  
Weight updated to: 0.2  
Total error Value: 1  
Weight updated to: 0.30000000000000004  
Weight updated to: 0.30000000000000004  
Total error Value: 1  
Weight updated to: 0.4  
Weight updated to: 0.4  
Total error Value: 1  
Weight updated to: 0.5  
Weight updated to: 0.5  
Total error Value: 1  
Total error Value: 0
```

## ▼ Classification

```
# Now we have the final weights that will be used to classify new instances of the data after training.
```

```
weights
```

```
array([0.5, 0.5])
```

```
cal_output(np.array([0,0]))
```

```
0
```

```
cal_output(np.array([0,1]))
```

```
0
```

```
cal_output(np.array([1,0]))
```

```
0
```

```
cal_output(np.array([1,1]))
```

```
1
```

## ✓ Conclusion

The perceptron model was successfully implemented and the different stages of the algorithm, including input definition, weight initialisation, training and prediction were reviewed. The model was tested using all four possible input combinations for the AND logic gate: (0,0), (0,1), (1,0), and (1,1). In each case, the predicted output matched the expected result, demonstrating that the perceptron successfully learned the AND operation and correctly classified every possible input.

Start coding or [generate](#) with AI.